Sensor Ablation Experiments: Detailed Code Walkthrough and Results

Project: Students Ergonomics Monitoring

June 14, 2026

Abstract

This document explains, in detail, the two sensor ablation experiments used in the Students Ergonomics Monitoring project, and summarizes their results. The goal of a *sensor ablation* is to answer a deployment question: *if we remove one or two of the three sensors, how much classification performance do we lose?* The project fuses three sensing sources — a body camera (posture), a face camera (blink/eye behaviour), and a wearable (heart) — into 18 numerical features. Two scripts re-run the classification pipelines while systematically keeping only subsets of these sensors: `supervised_sensor_ablation_experiment.py` (Random Forest) and `unsupervised_sensor_ablation_experiment.py` (Gaussian Mixture clustering). Both use Leave-One-Subject-Out cross-validation so that the reported numbers reflect generalization to subjects never seen in training. The headline result is that **Posture + Heart is a near-free replacement for the full sensor set**, while removing the Heart wearable is what costs the most when distinguishing sleepiness from stress.

1 Motivation and Shared Setup

1.1 Why a sensor ablation?

The full system relies on three independent sensing channels. In a real deployment one of them may be unavailable: the wearable may not be worn, the face may be out of the camera frame, or only a single camera may be present. An ablation study quantifies the cost of each such degradation *before* it happens in the field, so we can decide which sensors are essential and which are redundant. Concretely, for every reduced sensor set the experiments measure how far performance drops relative to the full 18-feature baseline.

1.2 The three sensor groups (18 features)

Both scripts partition the 18 features identically into three sensor groups (Table 1).

The Posture features summarize shoulder symmetry, head tilt and forward-head-posture (mean, variability, and the percentage of time in a "bad" range). The Blink features summarize eye aspect ratio, PERCLOS (percentage of eye closure), blink count/frequency, and inter-blink interval. The Heart features are mean heart rate (`HR`) and a heart-rate-variability measure (`RMMSD`).

Table 1: Sensor groups and their features.

Sensor	#	Features
Posture (body camera)	9	`p_shoulder_mean/std/bad_pct`, `p_head_tilt_mean/std/bad_pct`, `p_fhp_mean/std/bad_pct`
Blink (face camera)	7	`b_ear_mean/std`, `b_perclos`, `b_blink_count`, `b_blink_freq`, `b_ibi_mean/std`
Heart (wearable)	2	`HR`, `RMMSD`

1.3 The six ablation configurations

Each experiment evaluates six reduced sensor sets: the three *pairs* (keep two, drop one) and the three *singles* (keep one, drop two):

- `keep_posture_blink` (drop Heart), `keep_posture_heart` (drop Blink), `keep_blink_heart` (drop Posture);
- `keep_posture`, `keep_blink`, `keep_heart`.

A seventh "full" configuration with all 18 features is the reference baseline.

1.4 The three evaluation scenarios

For every configuration both scripts compute up to three scenarios, which is the conceptual core of the experiment:

`full` Train *and* test with all 18 features. Per-task reference baseline.

`clean` Train *and* test using only the kept-sensor features. This is the realistic "we simply do not have that sensor" setup — the model never sees the missing columns. This is the **trustworthy** number for the ablation.

`masked` Train with all 18 features, then at test time set the excluded columns to `NaN` so the training-fold imputer fills them with the training *median*. This is a deployment-mismatch stress test: a model trained expecting all sensors but run with some missing. Reported only for analysis.

The clean/masked distinction matters because a model trained on a feature and then fed a constant median for it can behave pathologically (see results), whereas a model trained without that feature degrades gracefully.

1.5 Leave-One-Subject-Out (LOSO) cross-validation

Both scripts evaluate with `LeaveOneGroupOut` on the group column `video_source`, i.e. each subject/recording is held out in turn while the model is trained on all the others. Crucially, **all fitting happens inside the fold on the training subjects only** (imputer, scaler, model, and — for clustering — the cluster→label mapping). This prevents leakage and makes the reported metrics an honest estimate of performance on a *new* student.

2 Supervised Experiment (Random Forest)

File: `Scripts/ML/supervised_sensor_ablation_experiment.py`.

2.1 Model

A single model is used everywhere: a scikit-learn `Pipeline` of `SimpleImputer(strategy="median")` followed by a `RandomForestClassifier` (retrieved from the project's shared `model_factories`, fixed `random_state=42`). Random Forest was the best supervised baseline in the project, so the ablation isolates the effect of *sensors*, not of model choice.

2.2 The four classification tasks

The script reuses the dataset builders from `wellbeing_hierarchical_classification` and runs four tasks:

1. **binary**: *normal* vs *abnormal*.
2. **subtype**: *sleepy* vs *stress* (only abnormal rows).
3. **hierarchical**: a two-stage classifier — first binary (normal/abnormal), then on the abnormal predictions a subtype stage (sleepy/stress). Both stages are Random Forests ("RF → RF").
4. **direct_3class**: a flat three-way classifier (normal/sleepy/stress).

The hierarchical and direct 3-class tasks solve the same end problem two different ways and are therefore directly comparable.

2.3 How the masking is implemented

The function `mask_features_at_test` copies the test matrix and overwrites the excluded-sensor columns with `NaN`. Because the pipeline's imputer was fit on the training fold, those `NaN`s become the *training* median at predict time — this is exactly the deployment-mismatch scenario. In the `clean` scenario, instead, the columns are never present: the pipeline is trained only on `keep_features`.

2.4 Flat task evaluation

`evaluate_flat_loso` runs the standard LOSO loop for the binary, subtype and direct 3-class tasks: for each held-out subject it clones the pipeline, fits on the training rows, optionally masks the test columns, predicts, and records per-fold metrics. It also accumulates all out-of-fold predictions to produce one pooled ("overall") metric row and one pooled confusion matrix per task.

2.5 Hierarchical task evaluation

`evaluate_hierarchical_loso` is more involved. Per fold it trains *two* Random Forests on the training subjects: a binary model and a subtype model. On the held-out subject it first predicts normal/abnormal; rows predicted *normal* are finalized as normal, and only the rows predicted *abnormal* are passed to the subtype model to be labelled sleepy or stress. This mirrors the real decision flow "is something wrong? if so, what kind?".

2.6 Metrics and outputs

Each fold and the pooled overall row report macro-F1, balanced accuracy, accuracy and weighted-F1 (via the shared `compute_metrics`). The script writes, under `results/supervised/sensor_ablation/`: a `metrics.csv` per configuration, pooled confusion-matrix PNGs for full/clean/masked of every task, and a single human-readable `sensor_ablation_report.txt`. The report also includes an automatic *interpretation* section that flags, per configuration, where dropping the sensors hurts macro-F1 the most (≤ -0.05), where it barely matters, where it surprisingly helps, and whether `clean` or `masked` is better.

3 Unsupervised Experiment (Gaussian Mixture)

File: `Scripts/ML/unsupervised_sensor_ablation_experiment.py`.

3.1 Model and pipeline

The unsupervised counterpart uses a `GaussianMixture` (`covariance_type="full"`, the best unsupervised baseline in the project), preceded by `SimpleImputer(median)` + `StandardScaler`. Standardization matters here because a Gaussian Mixture is distance/covariance based, unlike the tree model. It runs two tasks: **binary** (`n_components=2`) and **direct_3class** (`n_components=3`). There is no hierarchical or subtype-only task on the unsupervised side.

3.2 The clustering-to-label challenge

A clustering algorithm produces anonymous cluster indices, not class names. The key routine `run_gmm_loso` solves this honestly under LOSO:

1. Fit the imputer+scaler+GMM on the *training subjects only*.
2. Derive a `cluster`→`label` mapping by *majority vote*: each cluster is named after the most frequent true label among the training rows assigned to it. A component that received no training rows falls back to the globally most frequent training label (so test predictions can never raise a `KeyError`).
3. *Freeze* that mapping and apply it to the held-out subject's cluster assignments.

This deliberately mirrors how the offline-trained GMM in the project's real-time GUI labels live data from a subject it has never seen. The same `clean`/`masked`/`full` scenarios are evaluated, with masking again implemented by setting excluded columns to `NaN` so the training-fold median is imputed.

3.3 Metrics and outputs

Using the project's `classification_metrics`, it reports the same metric family (macro-F1, balanced accuracy, accuracy, weighted-F1) plus confusion matrices, and writes everything under `results/unsupervised/sensor_ablation/` (per-configuration `metrics.csv`, a `sensor_ablation_summary.c` confusion-matrix PNGs, and a text report with the same auto-interpretation logic).

4 Results

4.1 Full-sensor reference baselines

Table 2 gives the all-18-feature reference. Supervised performance is high across the board (subtype is essentially perfect), and is the standard against which every ablation delta is measured. The unsupervised baselines are lower, as expected for a method that never sees labels during fitting.

Table 2: FULL (18-feature) reference, macro-F1 / balanced accuracy.

Task	Supervised (RF)	Unsupervised (GMM)
binary	0.952 / 0.958	0.877 / 0.857
subtype	1.000 / 1.000	—
hierarchical	0.957 / 0.957	—
direct_3class	0.951 / 0.951	0.700 / 0.718

4.2 Supervised ablation (clean scenario, macro-F1)

Table 3 reports the `clean` macro-F1 for each configuration with its delta versus the full baseline. The clean scenario is the trustworthy one (the model is never told about missing sensors).

Table 3: Supervised CLEAN macro-F1 (Δ vs full). Bold = best non-full configuration per task.

Config (#feat)	binary	subtype	hierarchical	direct_3class
Posture+Blink (16)	0.954 (+.00)	0.693 (−.31)	0.763 (−.20)	0.750 (−.20)
Posture+Heart (11)	**0.957** (+.01)	**0.998** (−.00)	**0.960** (+.00)	**0.959** (+.01)
Blink+Heart (9)	0.859 (−.09)	0.981 (−.02)	0.864 (−.09)	0.879 (−.07)
Posture only (9)	0.950 (−.00)	0.579 (−.42)	0.697 (−.26)	0.699 (−.25)
Blink only (7)	0.862 (−.09)	0.604 (−.40)	0.654 (−.30)	0.661 (−.29)
Heart only (2)	0.802 (−.15)	0.988 (−.01)	0.829 (−.13)	0.836 (−.12)

Key observations (supervised):

- **Posture + Heart is essentially free.** Dropping the face/Blink camera keeps every task within ±0.01 of the full baseline (and even slightly improves direct_3class). With only 11 features it matches the full system.
- **Heart is the decisive sensor for sleepy-vs-stress.** Every configuration that *drops* Heart collapses on the subtype task (Posture+Blink −0.31, Posture only −0.42, Blink only −0.40), and this propagates to the hierarchical and 3-class tasks. Conversely, the 2 Heart features alone already solve subtype almost perfectly (0.988).
- **Posture is the backbone of the binary "something is wrong" decision.** Posture alone keeps binary at 0.950, and any set that drops Posture loses ∼0.09 on binary.
- **Complementarity.** Heart separates the *type* of abnormality; Posture detects *that* there is one. Together (Posture+Heart) they reconstruct the full system; neither alone does.
- **Clean beats masked** on the hard tasks: training without a sensor degrades gracefully, while median-imputing a sensor the model was trained on can be catastrophic — e.g. Blink+Heart

masked binary drops to 0.399 (the classifier predicts everything "abnormal") versus 0.859 clean.

4.3 Unsupervised ablation (clean scenario, macro-F1)

Table 4: Unsupervised CLEAN macro-F1 (Δ vs full).

Config (#feat)	binary	direct_3class
Posture+Blink (16)	0.969 (+.09)	0.671 (−.03)
Posture+Heart (11)	0.947 (+.07)	0.687 (−.01)
Blink+Heart (9)	0.781 (−.10)	0.647 (−.05)
Posture only (9)	0.953 (+.08)	0.589 (−.11)
Blink only (7)	0.804 (−.07)	0.550 (−.15)
Heart only (2)	0.378 (−.50)	0.483 (−.22)

Key observations (unsupervised):

- **Posture drives unsupervised separation.** Any configuration containing Posture reaches ≥ 0.95 on binary; dropping Posture (Blink+Heart, Blink only, Heart only) drops it sharply.
- **Removing sensors can help the GMM on binary.** Fewer, more informative dimensions (e.g. Posture-only or Posture+Blink) actually *improve* the binary macro-F1 over the full 18-feature mixture (+0.08/+0.09). The extra Heart/Blink noise destabilizes the unsupervised clusters in the full set.
- **Heart alone is useless without labels.** Two raw features cannot form meaningful normal/abnormal clusters (binary 0.378). This contrasts with the supervised case, where labelled Heart is very strong for subtype — underlining that Heart's value is in *supervised* fine discrimination, not unsupervised geometry.
- **Clean > masked** here too: median-imputing dropped sensors consistently underperforms simply re-fitting on the kept sensors.

4.4 Overall conclusion

Across both paradigms the same engineering message emerges:

1. **Posture is mandatory** — it carries the "normal vs abnormal" signal in both the supervised and unsupervised settings.
2. **Heart is the cheapest high-value add** — only two features, but it is what makes sleepy-vs-stress separable; **Posture + Heart (11 features) reproduces the full system** at essentially no cost.
3. **The Blink/face camera is the most droppable sensor** in the supervised pipeline, which is attractive for privacy and for single-camera deployments.
4. **Always retrain on the available sensors (clean), never median-impute a missing one (masked)** — the masked scenario can fail catastrophically and should be avoided in deployment.